



6. Time Complexity

"Measures the number of steps the algorithm has to make"

Previous Section:

 [5. Recursion](#)

Contents:

[1. What is Time Complexity](#)

[2. Time Complexity and Runtime](#)

[3. Big-O Notation](#)

[4. One Operation](#)

[5. Best Case, Average Case, Worst Case](#)

[6. Applications of Time Complexity](#)

[7. Space Complexity](#)

[Summary](#)

[References](#)

So far, we have explored and understood some of the most common algorithms in Computer Science such as Greedy Algorithms, Divide and Conquer, Backtracking, and Dynamic Programming. Each of them has its own strengths, weaknesses, and practical applications in different fields.

From this point onward, we will begin learning about important data structures including Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, and Hash Tables. However, before we dive into those topics, there is one extremely important concept that I must introduce first: **Time Complexity**.

Whenever we create or analyze an algorithm, we need a way to measure how efficient it is. Some algorithms solve problems very quickly, while others may take too much time when the amount of data becomes large. Time Complexity helps us estimate and compare the efficiency of algorithms by analyzing how the number of operations grows as the input size increases.

Understanding Time Complexity is essential because it allows us to:

- Predict how fast an algorithm performs.

- Compare multiple solutions to the same problem.
- Choose the most efficient approach for large-scale applications.
- Understand why some algorithms are preferred over others.

As we continue learning data structures and algorithms, Time Complexity will appear constantly. Therefore, having a strong understanding of this concept is extremely important.

1. What is Time Complexity

Time Complexity is defined as the number of steps required to execute an algorithm. Each step may involve comparisons, computations, function calls, iterations, memory access, and other operations that contribute to the total running time.

To make this easier to understand, let us use a simple real-life example. Imagine there is a classroom with 100 students, and one of them is secretly holding a pen. I need to find out who has the pen without knowing the answer beforehand. There are several possible methods I could use:

- **Method 1 - Asking Every Student Individually:** I go to each student one by one and ask whether they have the pen. If there are 100 students, I may need to ask 100 times in the worst case. This process grows directly with the number of students, so the Time Complexity is: $O(n)$ where n represents the number of students.
- **Method 2 - Asking Every Student About Everyone Else:** I ask the first student whether they have the pen, and then I ask them about all other students. After that, I repeat the same process for the next student. If there are 100 students, asking each student takes 100 operations. Asking about every other student also takes 100 operations for each student. The total work becomes approximately $O(n^2)$. This type of complexity grows very quickly as the number of students increases.
- **Method 3 - Dividing the Classroom into Groups:** Instead of checking students one by one, I divide the classroom into two groups and ask *"Is the student holding the pen on the left side or the right side?"*. After identifying the correct side, I divide that group again into two smaller groups and repeat the process until only one student remains. This method repeatedly cuts the problem size in half, producing a Time Complexity of $O(\log n)$. This is much faster for large amounts of data.

Choosing which method to use depends heavily on the situation:

- I may need to use $O(n)$ if only the student holding the pen knows the answer.
- I may use $O(n^2)$ if students can only provide information about other students individually.
- I may use $O(\log n)$ if everyone knows the answer and can guide me toward the correct group.

This is the core idea behind Time Complexity: ***Different algorithms solve the same problem using different numbers of steps, and some approaches are significantly more efficient***

than others.

2. Time Complexity and Runtime

Many beginner programmers often misunderstand the difference between **Runtime** and **Time Complexity**. Although they are related, they are not the same concept.

Runtime

Runtime refers to the actual amount of time required for a program or algorithm to execute on a computer. The runtime of an algorithm can change depending on many factors such as:

- The programming language used
- The operating system
- The hardware and CPU speed
- The compiler or interpreter
- The size of the input data
- The way the program is implemented

For example, if I recreate a Resonator using one invention and another inventor recreates the same Resonator using different technology, both systems may require different actual execution times even though they perform the same task. One machine may process the data faster, while another may process it slower.

Because runtime depends heavily on external conditions, it is not always the best way to compare algorithms.

Time Complexity

Time Complexity is a completely different concept. Instead of measuring actual execution time, it measures the number of operations or steps an algorithm performs as the input size increases.

When analyzing time complexity, I do not focus on seconds or milliseconds. Instead, I focus on how the algorithm behaves when the amount of data becomes larger.

For example:

- If observing one Resonator requires 10 actions, then observing 100 Resonators may require far more actions depending on the algorithm I use.
- Some methods increase slowly as the amount of data grows.
- Other methods become dramatically slower with large amounts of data.

Time Complexity allows me to estimate how scalable an algorithm is without caring about hardware specifications or programming languages.

Why Time Complexity is More Useful

Time Complexity is often preferred over Runtime when evaluating an algorithm because it provides a higher-level understanding of performance.

Unlike Runtime:

- Time Complexity does not depend on the computer being used.
- It does not depend on the programming language.
- It focuses on growth behavior as input size increases.
- It allows algorithms to be compared fairly across different implementations.

As an investigator and inventor, I care less about how fast a single experiment runs on one machine and more about whether the method remains efficient when the amount of information becomes massive. That is exactly what Time Complexity helps us understand.

3. Big-O Notation

The symbol (O) commonly seen in Time Complexity analysis is called **Big-O Notation**. In Mathematics, Big-O Notation is used to describe the upper bound of a function.

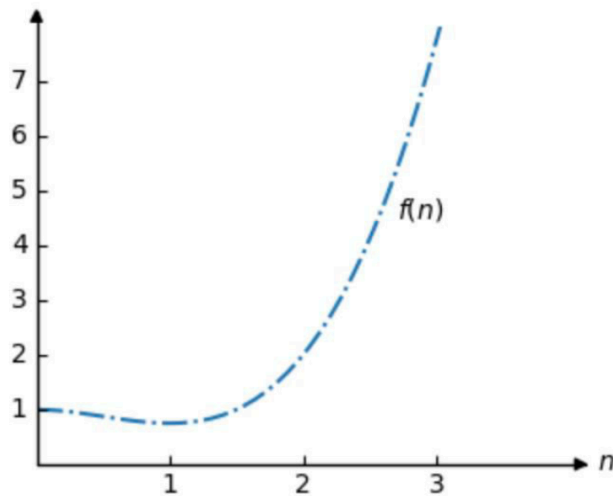
In Computer Science, Big-O Notation is used to describe the **worst-case growth rate** of an algorithm as the input size increases.

Big-O Notation is written as: $O(f(n))$, where: n usually represents the amount of input data; $f(n)$ represents how the number of operations grows.

Understanding Big-O Through Observation

Suppose I analyze an algorithm with the following function:

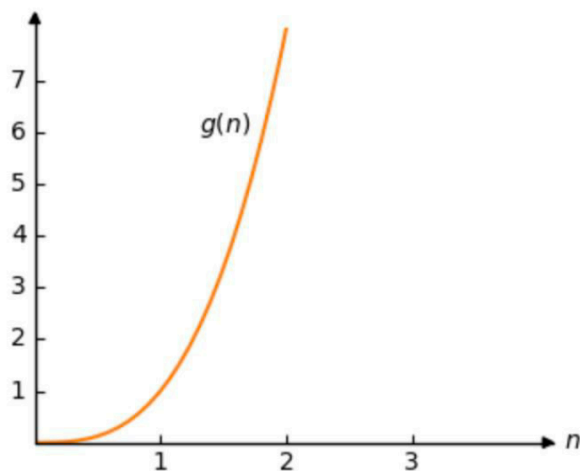
$$f(n) = 0.5n^3 - 0.75n^2 + 1$$



Source: [W3School](https://www.w3schools.com/graphics/js_functions_graph.asp)

Now consider another function:

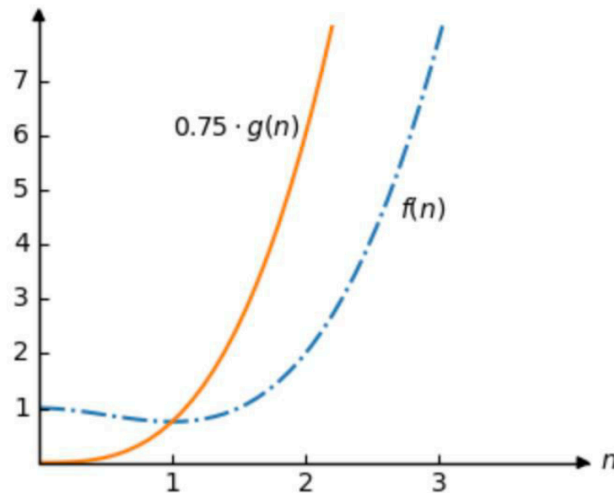
$$g(n) = n^3$$



Source: [W3School](https://www.w3schools.com/graphics/js_functions_graph.asp)

Although the first function contains multiple terms, the cubic term n^3 eventually dominates the behavior when n becomes very large.

We can observe this by looking at $0.75 * g(n)$ and $f(n)$ in the same plot:



Source: [W3School](https://www.w3schools.com/complexity/complexity.asp)

Because of that, we say the Time Complexity of the algorithm is: $O(n^3)$. This means the algorithm's growth behaves similarly to n^3 for large values of n .

Why Only Large Values Matter

When analyzing Time Complexity, I usually focus on very large amounts of data because that is when efficiency becomes important.

For example:

- Sorting 10 values is easy for almost any algorithm.
- Sorting 100 values is still manageable.
- But sorting millions of values can become extremely expensive depending on the algorithm used.

This is why Big-O Notation focuses on long-term growth rather than small inputs.

As the amount of data increases, inefficient algorithms become much harder to manage. Efficient algorithms, however, remain practical even at massive scales.

4. One Operation

When discussing Time Complexity, the word **operation** is an abstraction used to represent a basic action performed by an algorithm. An operation may take one or several CPU cycles internally, but for Time Complexity analysis, I simplify it into a single logical step.

Examples of operations include:

- Comparing two values
- Swapping two array elements

- Performing arithmetic calculations
- Accessing memory
- Assigning variables

Constant Time Operations

An operation is said to take **constant time** if it requires the same amount of work regardless of the input size.

For example, comparing two specific array elements takes approximately the same effort whether the array contains: 10 elements, 100 elements, 1000 elements. This is called: $O(1)$ or **constant time complexity**.

Example: Bubble Sort Operations

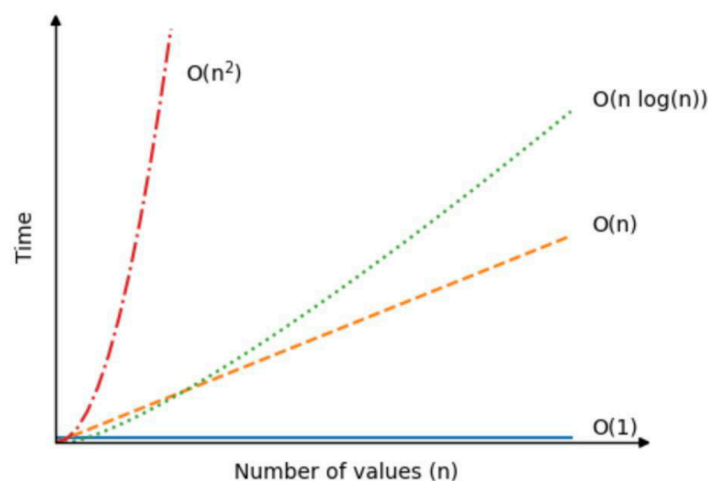
Consider the Bubble Sort algorithm. One step in Bubble Sort usually involves: Comparing two neighboring elements and swapping them if necessary

Even though this technically contains multiple small actions, I can treat the entire process as one operation because it always takes a constant amount of time. What truly matters is not the tiny details of each operation, but how many times those operations repeat as the data size grows.

Growth of Algorithms

Different algorithms grow at different rates:

- Some grow slowly and remain efficient.
- Others grow extremely fast and become inefficient for large datasets.



Source: [W3School](https://www.w3schools.com/time-complexity/)

That is why understanding operations and Time Complexity is essential. As I continue studying algorithms and data structures, Big-O Notation becomes the tool that helps me predict whether an approach is practical before I even execute the program.

5. Best Case, Average Case, Worst Case

When analyzing algorithms, I cannot simply say that one algorithm is “fast” or “slow” without considering the situation it operates in. Different inputs can drastically change how an algorithm behaves.

Because of that, algorithm analysis is commonly divided into three scenarios: Best Case; Average Case; Worst Case

These concepts help me understand how an algorithm performs under different conditions and different input arrangements.

Best-Case Time Complexity

The **best-case** scenario refers to the situation where the algorithm performs the **fewest possible operations** to produce the result.

Best-case analysis uses **Omega Notation: $\Omega(f(n))$** . Omega Notation represents the **lower bound** of the running time.

For example: If I search for a value in an array and the value happens to be at the first position, the search finishes immediately. The algorithm performs the minimum amount of work possible.

As an investigator, this is similar to immediately finding the clue I need at the very first location I inspect.

Average-Case Time Complexity

The **average-case** scenario measures the expected performance of an algorithm across all possible inputs.

Average-case analysis uses **Theta Notation: $\Theta(f(n))$** . Theta Notation represents the algorithm's **typical growth behavior**.

To calculate the average case: Consider all possible inputs → Compute the running time for every input → Sum all running times together. → Divide by the number of inputs.

This usually provides a more practical estimate of real-world performance.

For example: Some algorithmic movement patterns may be simple to analyze. Others may require more observation. Average-case analysis estimates how the system behaves across all observations overall.

Worst-Case Time Complexity

The **worst-case** scenario measures the maximum number of operations an algorithm may perform.

Worst-case analysis uses **Big-O Notation**: $O(f(n))$. Big-O represents the **upper bound** of an algorithm's running time.

This is the most commonly used form of Time Complexity analysis because it guarantees that: The algorithm will never perform worse than this limit. I can predict the maximum cost of the algorithm even in difficult situations.

As someone constantly working with incomplete information and massive amounts of investigative data, I rely heavily on worst-case analysis because I need guarantees, not optimistic assumptions.

Why Worst-Case Analysis is Preferred

Among the three analyses, worst-case analysis is generally the most important. This is because the best-case analysis may rarely happen, the average-case analysis can be difficult to calculate accurately. Some algorithms may require evaluating an enormous number of possible inputs. The Worst-case analysis provides a safer and more reliable benchmark.

Additionally, algorithm performance often depends more on the **input data** than the algorithm itself.

Example - Linear Search vs Binary Search

Suppose I have the following list of numbers and I want to find the value 3:

1, 2, 5, 6, 8, 9, 10, 13, 3, 7, 12, 11, 14, 4, 15

If I use **Linear Search**, I inspect values one by one until I finally discover 3 at index 8. The Time Complexity for Linear Search is: $O(n)$

Now suppose I sort the list first:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15

Because the data is now organized, I can use **Binary Search** and immediately narrow down the search space by repeatedly dividing the list in half. The value 3 is quickly found at index 2. Binary Search has a Time Complexity of $O(\log n)$.

This demonstrates how the structure of the input data can heavily affect algorithm performance.

Importance of Different Cases

Although worst-case analysis is commonly preferred, best-case and average-case analyses are still valuable because they provide additional insight into how algorithms behave under specific conditions.

Together, these three perspectives help me better understand: Algorithm reliability;
Scalability; Efficiency under different input patterns

6. Applications of Time Complexity

Different algorithms grow at different rates as the amount of input data increases.

Below is the general ascending order of common Time Complexities:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(10^n) < O(n!)$$

Algorithms near the left side are generally more efficient than algorithms near the right side.

Common Examples of Time Complexities

- **Constant Time — $O(1)$:** Algorithms that always perform the same amount of work regardless of input size. Examples:
 - Accessing an array index
 - Push and Pop operations in a Stack
 - Enqueue and Dequeue operations in a Queue
 - Accessing parent or child nodes in a Tree

As an inventor, this resembles instantly retrieving a specific blueprint from a labeled storage system.

- **Linear Time — $O(n)$:** Algorithms whose operations grow directly with the amount of data. Examples:
 - Traversing an Array
 - Traversing a Linked List
 - Comparing two strings
 - Linear Search

This resembles manually observing every person one by one during an investigation.

- **Logarithmic Time — $O(\log n)$:** Algorithms that repeatedly divide the problem into smaller parts. Examples:
 - Binary Search

- Divide and Conquer algorithms
- Operations in Binary Search Trees

This resembles narrowing down suspects by repeatedly eliminating half of the possibilities.

- **Linearithmic Time** — $O(n \log n)$: Highly efficient sorting algorithms often fall into this category. Examples:
 - Merge Sort
 - Heap Sort
 - Quick Sort (average case)

These algorithms balance division and processing efficiently.

- **Quadratic Time** — $O(n^2)$: Algorithms involving nested iterations over data. Examples:
 - Traversing a 2D Array
 - Bubble Sort
 - Insertion Sort
 - Selection Sort

This resembles comparing every person against every other person individually.

- **Factorial Time** — $O(n!)$: Extremely expensive algorithms where possibilities grow explosively. Examples:
 - Traveling Salesman Problem using Brute Force
 - Generating all unrestricted permutations
 - Enumerating all partitions of a set

Such algorithms quickly become impractical even for relatively small inputs.

7. Space Complexity

While Time Complexity measures the amount of work an algorithm performs, **Space Complexity** measures the amount of memory an algorithm requires. Space Complexity is also expressed using Big-O Notation.

Space Complexity vs Auxiliary Space

Many learners confuse: **Space Complexity and Auxiliary Space**. They are related, but different.

- Auxiliary Space refers to the **extra temporary memory** used by an algorithm during execution.
- Space Complexity refers to the **total memory usage**, including: Input storage; Auxiliary memory; Temporary variables; Additional data structures

Why Space Complexity Matters

Memory efficiency is extremely important in algorithm design. Understanding Space Complexity helps with: Resource Management; Performance Optimization; Algorithm Selection; Overall Efficiency.

Some systems may have limited memory resources, meaning even fast algorithms can become impractical if they consume too much space.

As someone constantly processing observations, recordings, and recreations, memory management becomes just as important as execution speed.

Relationship Between Time and Space

Sometimes faster algorithms use more memory, while memory-efficient algorithms may take longer to execute.

Because of this, algorithm design often requires balancing: Time Efficiency and Space Efficiency. An algorithm is truly effective only when it manages both properly.

Common Space Complexities - Examples:

- $O(1)$ Constant extra memory
- $O(n)$: Memory grows linearly with input size
- $O(\log n)$: Recursive call stack growth
- $O(n + k)$: Additional counting or bucket structures

The following table summarizes the Time Complexity and Space Complexity of several common algorithms:

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Searching					
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	-
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	-
Sorting					
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Graph					
BFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$	-
DFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$	-
Dijkstra	$O((V+E) \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$	-
Floyd-Warshall	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^2)$	-
Kruskal	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$	$O(V)$	-
Prim	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$	$O(V)$	-

Source: [Data Structures & Algorithms: The Complete Guide | by Nilesh Verma | Medium](#)

The following table summarize Time Complexity and Space Complexity:

Aspect	Time Complexity	Space Complexity
What it measures	Execution time	Memory usage
Affected by	Loops, recursion	New variables, recursion
Example	$O(n^2) = 100$ steps	$O(n^2) = 100$ elements

Source: <https://imvishalpandey.medium.com/what-does-space-complexity-mean-in-plain-english-3303cff6463c>

Summary

Time Complexity measures the number of operations required to execute an algorithm, while Runtime measures the actual execution time on a machine. These are different concepts.

Time Complexity is valuable because it helps me understand how algorithm performance scales as input size increases, independent of hardware or programming language.

Big-O Notation provides a way to classify algorithm growth and estimate worst-case performance.

Space Complexity measures the amount of memory required by an algorithm and is equally important in algorithm analysis.

Together, Time Complexity and Space Complexity form the foundation of understanding algorithm efficiency, scalability, and practical performance in real-world applications.

References

<https://www.geeksforgeeks.org/dsa/understanding-time-complexity-simple-examples/>

<https://www.mygreatlearning.com/blog/why-is-time-complexity-essential/>

Next Section:

 [7. Arrays](#)